códigofacilito

Clase 9 • Bootcamp AI Engineer

De Workflows a Agentes

Pipelines declarativos, herramientas y control de ejecución

códigofacilito

Recap: Dónde estamos

Hasta ahora hemos construido tres niveles de complejidad creciente. Hoy cruzamos del Nivel 2 al Nivel 3.

- **Nivel 1 — Prompt Engineering:** El humano escribe un prompt, el LLM responde. Flujo lineal: entrada → modelo → salida.
- **Nivel 2 — RAG:** Añadimos contexto externo. El flujo sigue lineal: pregunta → retrieval → prompt enriquecido → respuesta.
- **Nivel 3 — Agentes:** El LLM decide qué hacer. Razona, elige herramientas y ejecuta acciones. El flujo deja de ser lineal.
-
-

¿Por qué RAG no es suficiente?

RAG resuelve un problema concreto: darle al modelo información que no tiene. Pero es un pipeline fijo. Siempre ejecuta los mismos pasos en el mismo orden: chunk → embed → retrieve → generate. ¿Qué pasa cuando la tarea no es una simple pregunta-respuesta?

Ejemplo: Un usuario pregunta "¿Cuál es el estado de mi pedido #4521 y, si está retrasado, envía un correo al proveedor?" Con RAG puro podemos buscar info del pedido, pero no podemos: consultar una API, evaluar si hay retraso, decidir si enviar correo, redactarlo y enviarlo. Cada acción requiere una decisión condicional. El flujo ya no es lineal, es dinámico.

códigofacilito

Espectro de autonomía

Podemos pensar en un espectro de autonomía con cuatro niveles, desde control total del programador hasta autonomía del modelo.

- **Código tradicional:** if/else escritos por el programador. Cero autonomía del modelo. 100% predecible.
- **Pipeline/Workflow:** Secuencia de pasos predefinida donde el LLM ejecuta tareas específicas. El orden lo decide el programador.
- **Router condicional:** El LLM clasifica la entrada y elige entre rutas predefinidas. Hay cierta toma de decisiones, pero acotada.
- **Agente:** El LLM decide qué herramienta usar, cuándo usarla y cuándo detenerse. El desarrollador define herramientas y restricciones.
-

códigofacilito

Pipelines

Un pipeline es una secuencia ordenada de pasos donde cada paso recibe una entrada, la transforma y pasa el resultado al siguiente.

Nuestro RAG de clases anteriores es un pipeline: ingest → chunk → embed → store → retrieve → generate.

- **Determinista:** Dado el mismo input, ejecuta los mismos pasos en el mismo orden.
- **Trazable:** Puedes registrar qué ocurrió en cada paso: input, output, duración, errores.
- **Componible:** Puedes reutilizar pasos individuales en diferentes pipelines.
- **Testeable:** Cada paso se puede probar de forma aislada con unit tests.
-

códigofacilito

Agentes

Un agente es un sistema donde el LLM actúa como motor de decisiones. Observa, razona, ejecuta y decide si necesita hacer más.

La diferencia fundamental: en un pipeline, el desarrollador escribe el flujo; en un agente, el LLM descubre el flujo.

- **No determinista:** Dado el mismo input, puede tomar caminos diferentes según su razonamiento.
- **Iterativo:** Opera en un bucle hasta que decide que la tarea está completa.
- **Usa herramientas:** Puede invocar funciones externas: APIs, bases de datos, código, RAG.
- **Autónomo (con límites):** El desarrollador define herramientas y restricciones, pero el modelo decide el flujo.

códigofacilito

Pipeline vs. Agente

Pipeline

Flujo definido por el desarrollador.
Pasos fijos y predecibles.
Fácil de debuggear.
Costo y latencia controlados.
Ideal para tareas repetitivas.

Agente

Flujo definido por el LLM en runtime.
Pasos variables según razonamiento.
Más difícil de debuggear. Costo y latencia variables. Ideal para tareas complejas y ambiguas.

códigofacilito

¿Cuándo NO usar un agente?

Usa un Pipeline

Tarea con pasos conocidos y orden fijo.
Latencia baja y costos predecibles.

Usa un Agente

Cuando el usuario puede pedir cosas impredecibles, la tarea requiere múltiples herramientas variables, se necesita razonamiento multi-paso o el flujo depende de resultados intermedios.

códigofacilito

Tools

Una herramienta es una función que el agente puede invocar. Para que el LLM la use correctamente, necesita tres cosas:

El agente lee estos elementos y decide: “Necesito llamar a search_documents con query=estado del pedido”.

- **Nombre claro:** Un nombre descriptivo que indique qué hace. “search_documents” es mejor que “tool_1”.
- **Descripción precisa:** Descripción en lenguaje natural de cuándo y para qué usarla. Es parte del prompt del agente.
- **Schema de parámetros:** Qué argumentos acepta, sus tipos y cuáles son obligatorios. Conecta con los outputs estructurados de Clase 4.

códigofacilito

Tool Design

Una herramienta bien diseñada es atómica, clara y segura.

- **Atómica:** Hace una sola cosa. “buscar_y_enviar_correo” es mala; “buscar_pedido” y “enviar_correo” son dos buenas herramientas.
- **Clara en descripción:** Debe indicar cuándo usarla y cuándo no. Si dos herramientas se solapan, el agente se confundirá.
- **Parámetros mínimos:** Menos parámetros = menos posibilidad de error del LLM. Usa valores por defecto cuando sea posible.
- **Segura:** Nunca expongas herramientas destructivas sin validación. Borrar datos o enviar correos requiere confirmación.

códigofacilito

Errores

Con LLMs, los errores son estocásticos: la misma llamada puede funcionar una vez y fallar la siguiente.

- **Errores de infraestructura:** El API devuelve 429 (rate limit), 500 o timeout. Solución: reintentos con backoff exponencial.
- **Errores de formato:** El LLM devuelve JSON malformado. Solución: validación con Pydantic (Clase 4) + reintento con feedback.
- **Errores de razonamiento:** El LLM elige herramienta incorrecta o genera parámetros inválidos. Solución: validación previa.
- **Errores de herramienta:** La API externa falla. Solución: manejo de excepciones + respuesta legible para el agente.

Estrategia de reintentos

Backoff exponencial: Espera 1s, luego 2s, luego 4s, luego 8s entre reintentos.
Añade jitter (variación aleatoria) para evitar que múltiples clientes reintenten simultáneamente.
Un reintento ingenuo puede empeorar el rate limiting.

Reintentos con contexto:
Cuando el modelo genera una respuesta inválida, el reintento debe incluir el error en el prompt. Define un máximo de intentos (típicamente 3) y un timeout global. Si después de 3 intentos no funciona, falla gracefully.

códigofacilito

Timeouts

Un agente sin límites de tiempo o costo es un riesgo. Puede entrar en loops o generar costos descontrolados.

Estos límites no son opcionales en producción.

- **Timeout por paso:** Cada llamada al LLM o herramienta tiene un tiempo máximo. Si se excede, se cancela y registra el error.
- **Máximo de iteraciones:** Un agente ReAct opera en bucle. Define un máximo (ej: 10) para evitar loops infinitos.
- **Presupuesto de tokens:** Conectando con Clase 2, define un presupuesto máximo de tokens por ejecución.
- **Presupuesto de costo:** Traduce tokens a dinero. “Esta ejecución no debe exceder $0.50 USD.”

Si no puedes ver qué pasó, no puedes arreglar lo que falló

Cada paso debe registrar: qué entró (input), qué salió (output), cuánto tardó, cuántos tokens consumió, y si hubo errores. Sin trazabilidad, debuggear un sistema con LLM es como debuggear código sin logs ni stack traces.

En nuestro proyecto usaremos el sistema de logging de Clase 1 y lo extenderemos para registrar cada paso del pipeline como un “span” con metadata completa.

```
orchestration/ — Estructura general

project/
├── core/
│   ├── llm_client.py        # Clase 1
│   └── tokenlab.py           # Clase 2
├── prompting/
│   └── promptkit.py          # Clase 3
├── contracts/
│   └── structured_output.py  # Clase 4
├── rag/                      # Clases 5-8
│   ├── retrieval.py
│   └── vectorstore.py
├── orchestration/            # NUEVO
│   ├── __init__.py
│   ├── pipelines.py
│   └── tools.py
└── inference/
    └── local_adapter.py      # Clase 8
```

```
Paso 1 — StepResult: El bloque básico

@dataclass
class StepResult:
    output: Any
    duration_seconds: float
    tokens_used: int = 0
    cost_usd: float = 0.0
    error: Optional[str] = None
    metadata: dict = field(default_factory=dict)

    @property
    def success(self) -> bool:
        return self.error is None

# Contrato entre pasos: duración, tokens,
# costo y errores siempre disponibles.
```

Paso 2 — pipeline_step decorador

```
def pipeline_step(name, max_retries=2,
                  timeout_seconds=30.0):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs) -> StepResult:
            for attempt in range(1, max_retries+1):
                start = time.time()
                try:
                    result = func(*args, **kwargs)
                    duration = time.time() - start
                    logger.info(
                        f"[{name}] OK {duration:.2f}s")
                    return StepResult(
                        output=result,
                        duration_seconds=duration)
                except Exception as e:
                    if attempt == max_retries:
                        return StepResult(
                            output=None, error=str(e),
                            duration_seconds=...)
        return wrapper
    return decorator
```

Paso 3 — Pipeline Engine

```
class Pipeline:
    def __init__(self, name, steps):
        self.name = name
        self.steps = steps

    def run(self, initial_input) -> PipelineResult:
        results = []
        current = initial_input
        for step_fn in self.steps:
            result = step_fn(current)
            results.append(result)
            if not result.success:
                logger.error(
                    f"Pipeline '{self.name}' falló"
                    f" en '{step_fn.step_name}'")
                return PipelineResult(
                    steps=results,
                    final_output=None,
                    success=False, ...)
            current = result.output
        return PipelineResult(
            steps=results,
            final_output=current,
            success=True, ...)
```

```
Paso 4 — Tool Registry

@dataclass
class ToolDefinition:
    name: str
    description: str
    parameters: dict  # JSON Schema
    function: Callable
    requires_confirmation: bool = False

class ToolRegistry:
    def __init__(self):
        self._tools: dict[str, ToolDefinition] = {}

    def register(self, tool: ToolDefinition):
        self._tools[tool.name] = tool

    def get(self, name: str) -> ToolDefinition:
        if name not in self._tools:
            raise ValueError(f"Tool '{name}' no")
        return self._tools[name]

    def to_llm_format(self) -> list[dict]:
        return [{"name": t.name, ...}
                for t in self._tools.values()]
```

```
Paso 5 — Herramienta real (RAG como Tool)

def search_documents(query, top_k=5):
    results = hybrid_search(query, top_k)
    if not results:
        return "No se encontraron documentos."
    return "\n".join(
        f"[{i}] {d['content'][:300]}"
        for i, d in enumerate(results, 1))

rag_tool = ToolDefinition(
    name="search_documents",
    description=(
        "Busca info en documentos internos."
        " Usar para políticas y datos."),
    parameters={
        "type": "object",
        "properties": {
            "query": {"type": "string"},
            "top_k": {"type": "integer",
                      "default": 5}},
        "required": ["query"]},
    function=search_documents)
```

```
Paso 6 — RAG como pipeline trazable

@pipeline_step(name="retrieve", max_retries=2)
def retrieve_step(query):
    return hybrid_search(query, top_k=5)

@pipeline_step(name="format_context")
def format_context_step(docs):
    return "\n---\n".join(
        d["content"][:500] for d in docs)

@pipeline_step(name="generate", max_retries=2)
def generate_step(context):
    from core.llm_client import call_llm
    return call_llm(
        system="Responde usando SOLO contexto.",
        user=f"Contexto:\n{context}")

rag_pipeline = Pipeline(
    name="docops_rag",
    steps=[retrieve_step,
           format_context_step,
           generate_step])
result = rag_pipeline.run("¿Cuál es la política?")
```

códigofacilito

Clase 10: Agentes ReAct 🎯

Hoy construimos: Pipeline engine con trazabilidad y reintentos, Tool Registry y manejo de errores. En la Clase 10 construiremos un agente ReAct real. Tarea: Revisa orchestration/ y agrega una herramienta nueva al registry.